



DOSSIER PROFESSIONNEL (DP)

Nom de naissance ➤ BARBEY
Nom d'usage ➤ BARBEY
Prénom ➤ Mathieu
Adresse ➤ 3 rue du gros chêne, 28240, La Loupe

Titre professionnel visé

Développeur Web et Web Mobile

MODALITÉ D'ACCÈS :

- ☒ Parcours de formation
- ☐ Validation des Acquis de l'Expérience (VAE)

Présentation du dossier

Le dossier professionnel (DP) constitue un élément du système de validation du titre professionnel.

Ce titre est délivré par le Ministère chargé de l'emploi.

Le DP appartient au candidat. Il le conserve, l'actualise durant son parcours et le présente

obligatoirement à chaque session d'examen.

Pour rédiger le DP, le candidat peut être aidé par un formateur ou par un accompagnateur VAE.

Il est consulté par le jury au moment de la session d'examen.

Pour prendre sa décision, le jury dispose :

1. des résultats de la mise en situation professionnelle complétés, éventuellement, du questionnaire professionnel ou de l'entretien professionnel ou de l'entretien technique ou du questionnement à partir de productions.
2. du **Dossier Professionnel** (DP) dans lequel le candidat a consigné les preuves de sa pratique professionnelle.
3. des résultats des évaluations passées en cours de formation lorsque le candidat évalué est issu d'un parcours de formation
4. de l'entretien final (dans le cadre de la session titre).

[Arrêté du 22 décembre 2015, relatif aux conditions de délivrance des titres professionnels du ministère chargé de l'Emploi]

Ce dossier comporte :

- pour chaque activité-type du titre visé, un à trois exemples de pratique professionnelle ;
- un tableau à renseigner si le candidat souhaite porter à la connaissance du jury la détention d'un titre, d'un diplôme, d'un certificat de qualification professionnelle (CQP) ou des attestations de formation ;
- une déclaration sur l'honneur à compléter et à signer ;
- des documents illustrant la pratique professionnelle du candidat (facultatif)
- des annexes, si nécessaire.

Pour compléter ce dossier, le candidat dispose d'un site web en accès libre sur le site.



Sommaire

Exemples de pratique professionnelle

Développer le frontend d'une application web

p.

6

- Créer la maquette d'un site

p.

p.

6

- Créer une interface responsive

p.

p.

10

- Communiquer avec une API

p

p.

16

Développer le backend d'une application web

p.

19

- Créer une base de données relationnelles à partir d'un script SQL

p.

p.

19

- Développer le backend d'une application web

p.

p.

22

- Intégrer une solution de paiement dans une application web

p

p.

29

EXEMPLES DE PRATIQUE PROFESSIONNELLE

Activité-type 1 Développer le frontend d'une application web

Exemple n°1 - Créer la maquette d'un site

1. Décrivez les tâches ou opérations que vous avez effectuées, et dans quelles conditions :

La région Auvergne Rhône-Alpes m'a contacté car elle souhaite créer une plateforme pour mettre en lien plus facilement les artisans et les particuliers de la région. Elle veut plus précisément me donner comme mission la création des maquettes du site.

La région a pour contrainte que le site soit accessible à tous qu'importe l'âge ou le potentiel handicap tout en respectant la norme WCAG 2.1. Le site, pour respecter l'ère du temps, doit être responsive. Trois versions de chaque page seront à produire, la version mobile, la version tablette et la version desktop. De plus, l'identité graphique du site doit respecter l'identité graphique des autres sites de la région.

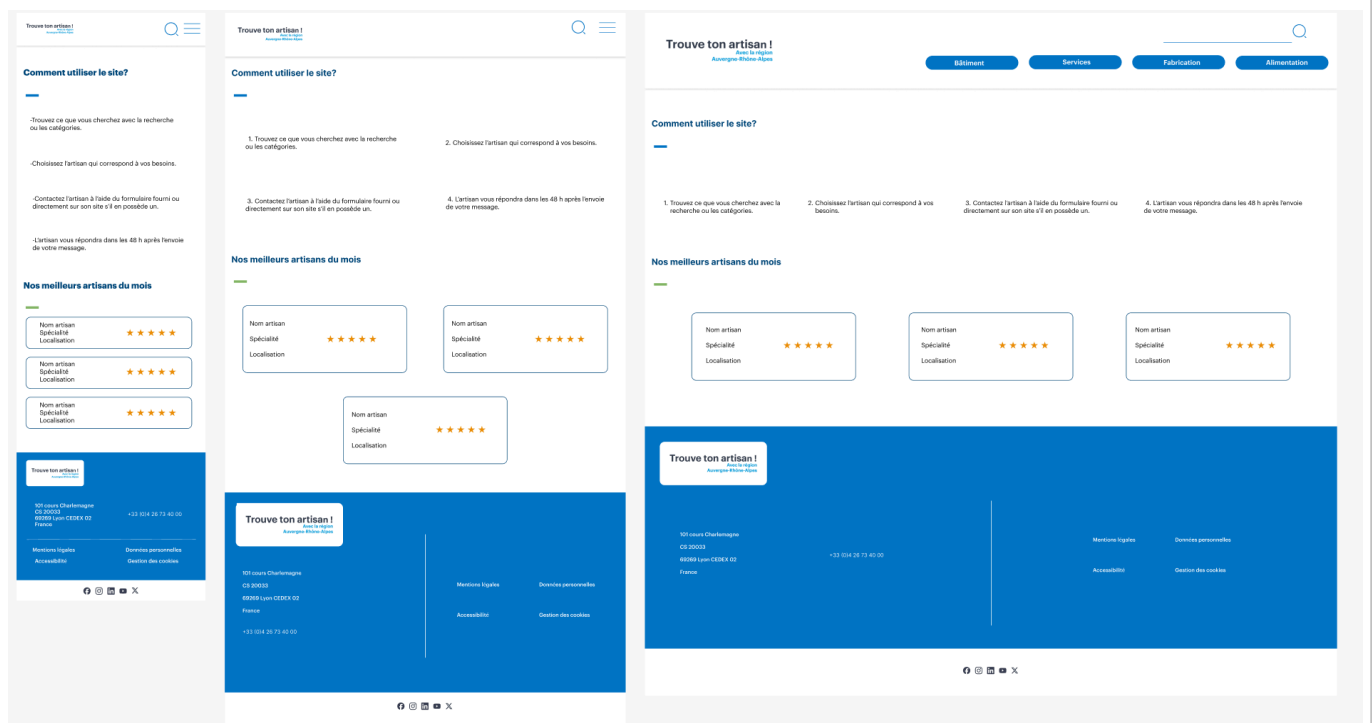
Pour garder cette identité, la région a imposé l'utilisation de la police "Graphik". La palette de couleur est définie comme telle:

#f1f8fc	#0074c7	#00497c	#384050	#cd2c2e	#82b864

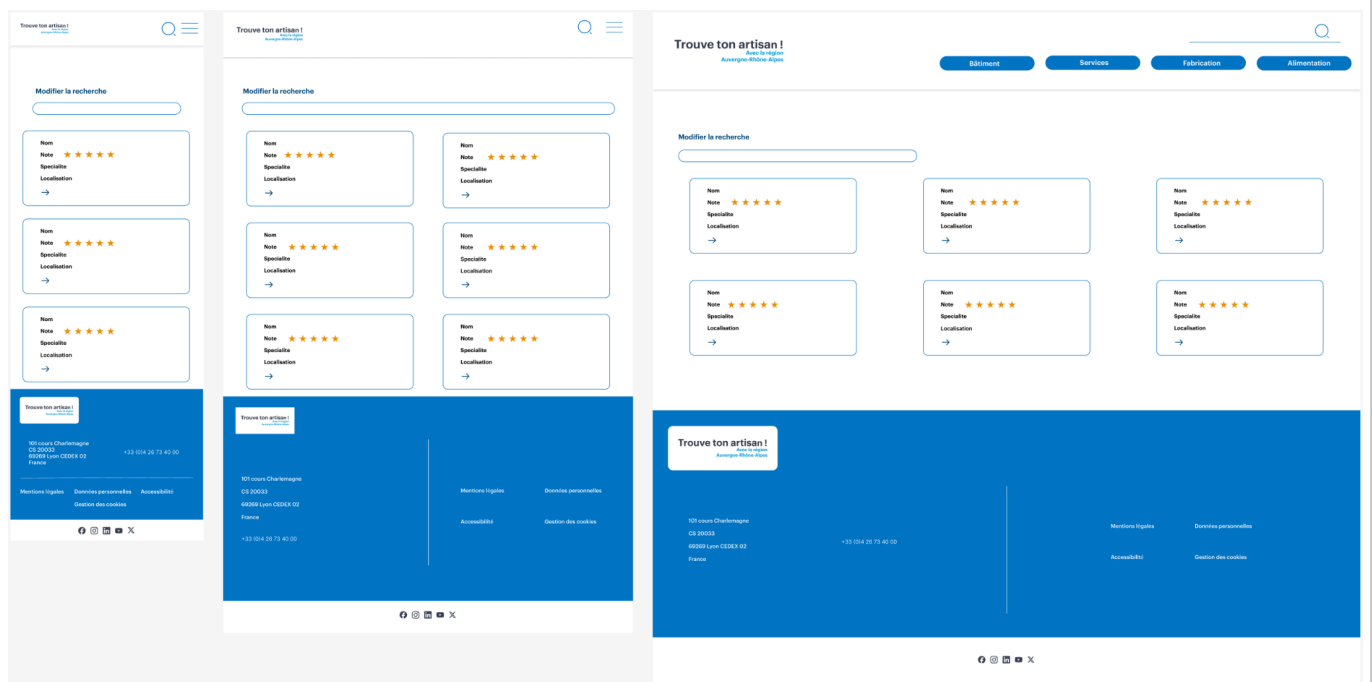
Trois pages ont été demandées: la page d'accueil, la page listant les artisans et la page artisan. Chaque page doit contenir un header et un footer. Le header doit contenir le logo qui, lorsqu'on clique dessus, ramène sur la page d'accueil. Il doit aussi contenir un menu avec les liens pour les pages des différents types d'artisans (Bâtiment, Services, etc), ainsi qu'une barre de recherche permettant de chercher les artisans par rapport à leur nom, leur spécialité ou la ville. Le footer, quant à lui, doit contenir un menu contenant les pages légales ainsi que l'adresse de l'antenne à Lyon.

Vous trouverez ci-dessous les maquettes de la page d'accueil en format mobile, tablette et desktop. La page d'accueil doit contenir deux sections: une section "Comment trouver mon artisan?" ainsi qu'une section "Nos meilleurs artisans du mois".

DOSSIER PROFESSIONNEL (DP)

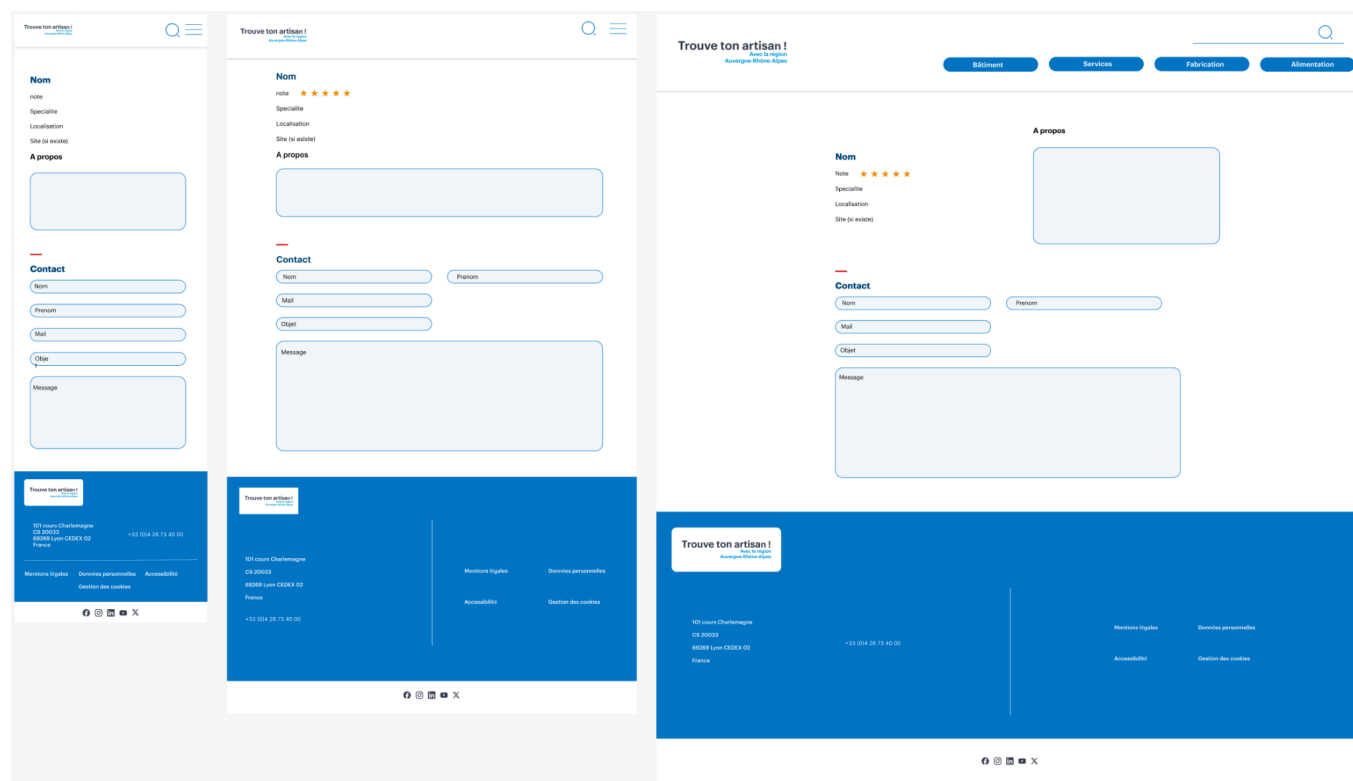


La seconde maquette est celle de la page contenant la liste des artisans. Cette liste doit contenir des fiches pour chaque artisan contenant le nom de celui-ci, sa note avec des étoiles, sa spécialité ainsi que sa localisation. Chaque fiche doit être cliquable pour rediriger vers la page de l'artisan.



DOSSIER PROFESSIONNEL (DP)

La dernière maquette est la page Artisan. Celle-ci représente ce que sera la page contenant toutes les informations des artisans. Elle doit contenir les éléments suivants: le nom de l'artisan, sa note visible avec des étoiles, sa spécialité, sa localisation, une rubrique "A propos" ainsi qu'un formulaire de contact ou un site web si celui-ci en a un.



2. Précisez les moyens utilisés :

Pour ce travail de maquettage, j'ai utilisé le logiciel Figma.

3. Avec qui avez-vous travaillé ?

J'ai travaillé seul.

4. Contexte

Nom de l'entreprise, organisme ou association ▶

La région Auvergne-Rhône-Alpes

Chantier, atelier, service ▶

Antenne de Lyon

DOSSIER PROFESSIONNEL ^(DP)

Période d'exercice ▶ Du : 01/07/2024 au : 13/10/2024

5. Informations complémentaires *(facultatif)*

Activité-type 1 Développer le frontend d'une application web

Exemple n° 2 - Créer une interface responsive

1. Décrivez les tâches ou opérations que vous avez effectuées, et dans quelles conditions :

Dans le cadre d'une candidature pour un poste de développeur Frontend React, j'ai décidé de créer mon propre CV à l'aide de ce même framework.

J'ai tout d'abord préparé mon fichier index.html, la racine de ce projet. Un fichier HTML se divise en deux parties: une partie header et une partie body. La partie header contient toutes les metadatas du site. Les balises <meta> vont contenir les informations tels que l'encodage des caractères, ici utf-8, la taille de la vue, ou encore la description du site. La balise <title> est dans la même veine en contenant le nom du site. Enfin, les balises <link> permettent de faire les liens vers des ressources extérieures comme ici avec le favicon du site.

```
<head>
  <meta charset="utf-8" />
  <link rel="icon" href="./icon/apple-touch-icon.png" />
  <meta name="viewport" content="width=device-width, initial-scale=1" />
  <meta
    name="description"
    content="Mon CV responsive créé avec React"
  />
  <title>Curriculum Vitae</title>
</head>
```

On retrouve ensuite la seconde partie d'un fichier HTML contenu dans la balise <body>. Dans un fichier index.html dans React, celle-ci ne contient qu'une balise <div> avec un id "root". C'est la porte d'entrée dans l'application et ce qui fait la force de React, le SPA ou Single Page Application.

```
<body>
  <div id="root"></div>
</body>
```

Après avoir préparé le point d'entrée de l'application, je me suis occupé du routing. React emporte une librairie appelée "react-router-dom" pour cela. Pour initialiser le routing de la page, il faut dans un premier temps créer la racine, le root de l'application. C'est là qu'intervient l'id="root" comme on le voit

dans la création de la constante root. Ensuite, la balise `<BrowserRouter>` va permettre l'utilisation des routes.

```
const root = ReactDOM.createRoot(document.getElementById('root'));
root.render(
  <React.StrictMode>
    <BrowserRouter>
      <App />
    </BrowserRouter>
  </React.StrictMode>
);
```

Ces routes sont définies dans le fichier App.js. Ce fichier est le contenu de la balise `<body>`. On y retrouve le header, le footer ainsi que le contenu de la balise `<main>`. Les routes sont définies dans la balise `<Routes>` par une balise `<Route>`. Elles portent comme attribut le path qui est l'url de la page, ainsi qu'un attribut élément qui va indiquer le nom qui a été attribué à chaque page pour la retrouver dans le projet, et l'afficher. Pour ce projet, j'ai décidé de créer six routes comme dans le code ci-dessous.

```
function App() {
  return (
    <div className="App bg-all">
      <Header />
      <main>
        <Routes>
          <Route path="/" element={<Home />} />
          <Route path="/service" element={<ServicePage />} />
          <Route path="/realisations" element={<Realisation />} />
          <Route path="/blog" element={<Blog />} />
          <Route path="/contact" element={<Contact />} />
          <Route path="/legalnotice" element={<LegalNotice />} />
        </Routes>
      </main>
      <Footer />
    </div>
  );
}
```

Comme on le voit sur l'extrait de code ci-dessus, certaines balises sont des balises orphelines comme la balise `<Header />`. Il s'agit d'un autre avantage des frameworks modernes comme React, les

composants. Grâce à cela, j'ai pu créer des composants que j'ai pu réutiliser dans le code comme le header ou encore les cards. On peut prendre en exemple le composant Header en exemple. A noter que j'ai utilisé des balises `<strong>` pour mettre en avant de le SEO des éléments tels que le nom du user.

```
function Header(props) {
  const location = useLocation()
  const [user, setUser] = useState([])
  const getUsers = async() => {
    const res = await fetch("https://api.github.com/users/github-john-doe")
    const json = await res.json()
    setUser(json)
  }

  useEffect(()=>{
    getUsers()
  },
  [])

  return (
    <Navbar activeKey={location.pathname} expand="lg" bg="headerBackground"
data-bs-theme="dark">
      <Container>
        <Navbar.Brand href="/" className="text-start"><strong
style={{fontSize: "1.5em"}}>{user.name}</strong></Navbar.Brand>
        <Navbar.Toggle aria-controls="basic-navbar-nav" />
        <Navbar.Collapse className="justify-content-end"
id="basic-navbar-nav" >
          <Nav activeKey={location.pathname}>
            <Nav.Link href="/"
className="text-uppercase">Accueil</Nav.Link>
            <Nav.Link href="/service"
className="text-uppercase">Service</Nav.Link>
            <Nav.Link href="/realisations"
className="text-uppercase">Realisations</Nav.Link>
            <Nav.Link href="/blog"
className="text-uppercase">Blog</Nav.Link>
            <Nav.Link href="/contact"
className="text-uppercase">Contact</Nav.Link>
          </Nav>
        </Navbar.Collapse>
      </Container>
    </Navbar>
  )
}
```

```
        </Navbar>
    )
}

export default Header
```

Note importante a propos de ce projet, bootstrap a été utilisé, mais j'ai aussi la librairie react-bootstrap. C'est de cette librairie que proviennent les balises telles que Navbar, Navbar.Collapse ou encore Nav.Link.

On observe d'abord une partie logique dans ce composant, récupérant ici la page active avec le `useLocation()`, ainsi que les informations de l'utilisateur qui est enregistré en format json. Ces informations sont utilisables dans la partie HTML du composant avec l'utilisation des `{}`. On remarque aussi le contenu des attributs `className` qui contient le style des balises, tel est le fonctionnement de bootstrap. Enfin, la dernière ligne permet de rendre disponible partout dans le code le composant.

Pour ce qui est du style de la page, j'ai utilisé bootstrap comme cité plus tôt. Celui-ci permet de gérer le positionnement directement dans les attributs des balises comme ci-dessous. `d-flex` permet d'activer le mode flexbox. Les éléments suivants permettent de gérer celui-ci. Ici, on indique que l'on veut que les éléments soient placés verticalement et qu'ils soient centrés verticalement et horizontalement.

```
<div className="bg-home d-flex flex-column justify-content-center align-items-center align-middle">
  <h1 className="home-title text-white text-center fw-semibold">
    Bonjour, je suis John Doe
  </h1>
  <h2 className="home-work text-white fw-semibold">
    Développeur web full stack
  </h2>
  <button onClick={scrollToAnchor} className="btn bg-button text-white" >
    En savoir plus
  </button>
</div>
```

Pour définir le responsive, bootstrap utilise le tag `col`. Celui-ci divise une page en douze colonnes. C'est pour cela qu'une balise contenant un tag `col-12` va être sur toute la longueur de la page. Pour créer ce qui s'appelle un breakpoint, c'est-à-dire indiquer à bootstrap à quel moment changer la disposition, on utilise des tags définis par bootstrap lui-même tels que `sm`, `md` ou encore `xl`. Chaque breakpoints a une longueur attribuée. C'est ainsi que le responsive fonctionne avec bootstrap comme on le voit dans l'extrait de code ci-dessous.

```
<div className="col-12 col-sm-6">
  <div className="mb-4 h2 fw-semibold border-bottom border-3 border-primary">
    A propos
  </div>
  <p className="fw-normal">
    Passionné par l'informatique et les nouvelles technologies, j'ai suivi une
    formation d'<strong>intégreteur-développeur web </strong> au CEF. Au cours de cette
    formation, j'ai pu acquérir des bases solides pour travailler dans le domaine du
    <strong>développement web</strong>.
  </p>
  <p className="fw-normal">
    Basé à Lyon, je suis en recherche d'une alternance au sein d'une agence
    digitale pour consolider ma formation de <strong>développeur web full stack</strong>.
  </p>
  <p className="fw-normal">
    J'accorde une attention particulière à la qualité du
    code que j'écris et je respecte les bonnes pratiques du web.
  </p>
</div>
```

Il est aussi possible de faire le responsive directement dans un fichier css avec les media query. Dans le cas de ce projet, il s'agit d'un fichier scss, bootstrap utilisant ce langage.. J'ai utilisé les media query pour gérer le responsive du background ainsi que changer l'image de celui-ci en fonction du format du support de l'utilisateur.

```
@media screen and (min-width: 576px) {
  .home-background{
    margin-bottom: 10%;
  }
  .bg-home{
    background-image: url("../style/img/hero-bg-tablette.jpg");
    background-color: rgba(0, 0, 0, 0.3);
    background-blend-mode: multiply;
    background-position-x: 80%;
    background-size: cover;
    background-repeat: no-repeat;
    height: 45vh;
  }
}
```

2. Précisez les moyens utilisés :

DOSSIER PROFESSIONNEL ^(DP)

Pour ce projet, j'ai utilisé le framework React, ainsi que le framework Bootstrap pour le style et le responsive. J'ai aussi utilisé les librairies React-Router-Dom pour le routing, ainsi que React-Bootstrap pour le header et la navbar.

3. Avec qui avez-vous travaillé ?

J'ai travaillé seul sur ce projet

4. Contexte

Nom de l'entreprise, organisme ou association ➤ Aucune

Chantier, atelier, service ➤ Recherche d'emploi

Période d'exercice ➤ Du : 24/03/2024 au : 03/04/2023

5. Informations complémentaires *(facultatif)*

Activité-type 1 Développer le frontend d'une application web

Exemple n° 3 - Communiquer avec une API

1. Décrivez les tâches ou opérations que vous avez effectuées, et dans quelles conditions :

Dans le cadre de ma formation, j'ai eu l'occasion de commencer à travailler sur un projet de boutique en ligne. Ce projet utilise une API REST faite avec NestJs pour communiquer avec la base de données. Il faut donc faire des calls vers cette API depuis le front. Sur ce projet, j'ai utilisé le framework NextJs, celui-ci étant basé lui-même sur React.

Il est possible de différencier différents calls. On va d'abord avoir les calls pour récupérer des informations de la base de données, ainsi que les calls pour créer, modifier ou supprimer des informations en base de données.

Le code ci-dessous est un exemple simple de call vers l'API pour récupérer des produits en base de données. On définit d'abord une constante en utilisant un hook useState. Un second hook est utilisé pour faire le call, le useEffect. On retrouve au sein de celui-ci un try/catch. Cela permet de retourner une erreur si le call ne fonctionne pas en indiquant l'origine de l'erreur à l'aide d'un statut. Si le call se passe sans encombre, on aura alors les catégories récupérer et disponible dans le composant NextJs.

```
const [categories, setCategories] = useState<Category[]>([]);

useEffect(() => {
  const fetchCategories = async () => {
    try {
      const res = await fetch(`${process.env.NEXT_PUBLIC_API}category/`, {
        cache: 'no-store'
      });

      if (!res.ok) throw new Error('Erreur lors de la récupération des catégories');

      const categoriesData: Category[] = await res.json();

      setCategories(categoriesData);
    } catch (error) {
      console.error('Erreur de chargement des catégories:', error);
    } finally {
      setLoadingCategories(false);
    }
  }
});
```

```
    }  
  };  
  
  fetchCategories();  
}, []);
```

Dans certains cas, les call API demandent plus de sécurité. Ayant mis en place une sécurité contre les attaques CSRF à l'aide d'un token CSRF, j'ai, dans les cas demandant un token CSRF, pu l'ajouter dans le headers. Il en est de même pour les cookies. La connexion étant gérée dans le backend avec un token JWT, celui-ci est stocké dans les cookies, encrypté. Certaines URLs demandent que l'utilisateur soit connecté pour être accessible.

```
async login(credentials: { mail: string; password: string }, csrfToken: string) {  
  try {  
    const res = await fetch(`${process.env.NEXT_PUBLIC_API}auth/login`, {  
      method: 'POST',  
      headers: {  
        'Content-Type': 'application/json',  
        'x-csrf-token': csrfToken,  
      },  
      credentials: 'include',  
      body: JSON.stringify(credentials),  
    });  
  
    const data = await res.json();  
  
    if (!res.ok) {  
      throw new Error(data.message || "Une erreur est survenue lors de la connexion.");  
    }  
  
    return data;  
  } catch (error: any) {  
    throw new Error(error.message);  
  }  
},
```

2. Précisez les moyens utilisés :

DOSSIER PROFESSIONNEL ^(DP)

Pour le frontend de ce projet, j'ai utilisé le framework NextJs ainsi que le framework Tailwind pour le style et le responsive.

3. Avec qui avez-vous travaillé ?

J'ai travaillé sur ce projet avec une personne elle-même en formation mais dans le domaine du marketing digital.

4. Contexte

Nom de l'entreprise, organisme ou association ▶ Sac'Azura

Chantier, atelier, service ▶ Boutique en ligne

Période d'exercice ▶ Du : 22/10/2025 au : actuellement en cours

5. Informations complémentaires *(facultatif)*

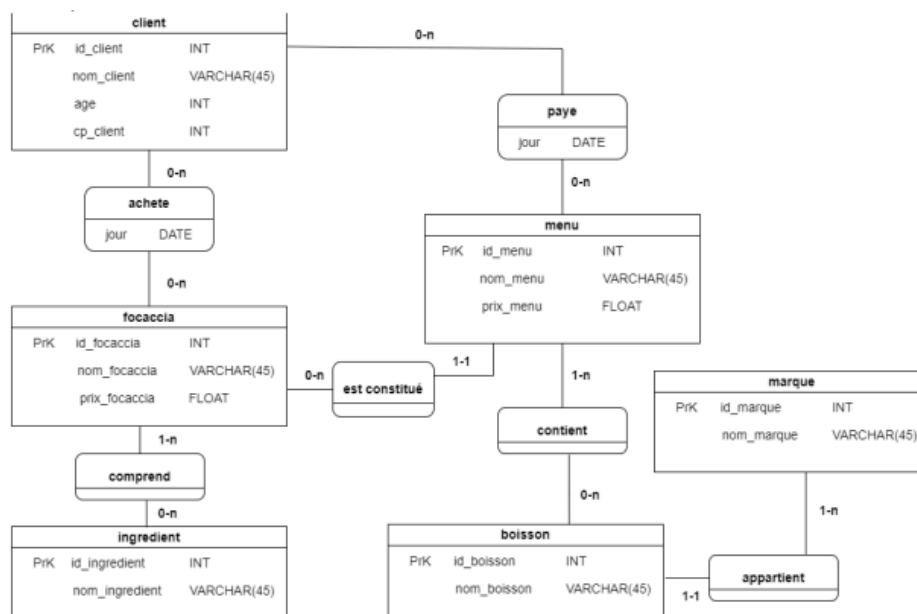
Le projet n'est pas encore arrivé à son terme. Il reste certains éléments du frontend à développer.

Activité-type 2 Développer le backend d'une application web

Exemple n° 1 - Créer une base de données relationnelle à partir d'un script SQL

1. Décrivez les tâches ou opérations que vous avez effectuées, et dans quelles conditions :

Le restaurant de street-food italien Tifosi m'a contacté pour la création d'une base de données qui sera hébergée localement afin d'avoir un meilleur suivi des commandes. La base de données doit suivre le modèle de conception ci-dessous.



On a six tables, reliées par des relations avec dans certains cas des tables ternaires. Si on décrit ce modèle de conception, on retrouve tout d'abord une table "menu". Cette table comporte trois relations, une relation 0-n avec la table "client", qui va traduire une relation Many-To-Many avec une table ternaire contenant le jour de la transaction. Elle contient aussi une relation One-To-Many avec la table "focaccia" traduisant qu'un menu ne peut avoir qu'une seule focaccia associée, mais qu'une focaccia peut être présente dans plusieurs menus. Quant à la relation avec la table "boisson", elle traduit aussi une relation Many-To-Many indiquant qu'un menu peut avoir plusieurs boissons, et qu'une boisson peut être dans plusieurs menus. Cette table "boisson" est liée à une table "marque" par une relation One-To-Many traduisant que les boissons ont une marque associée chacune, mais qu'une marque peut être associée à plusieurs boissons. Si on s'intéresse à la table "focaccia", on voit qu'elle est reliée à deux autres tables que la table "menu". Tout d'abord la table "ingrédient" qui est reliée par une relation

Many-To-Many, traduisant qu'une foccacia contient plusieurs ingrédients, et qu'un ingrédient peut-être utilisé dans plusieurs foccacia. Enfin, elle est reliée à la table "client" par une relation Many-To-Many et par une table ternaire contenant le jour de transaction.

Le restaurant a aussi demandé à ce qu'un compte avec mot de passe soit créé pour administrer la base de données.

Le script SQL commence comme tel ci-dessous. La première ligne sert à créer la base de données mais vérifie avant si celle-ci n'existe pas déjà. Le bloc suivant sert à créer le compte administrateur pour le restaurant, lui donnant tous les privilèges sur la base de données.

```
create database if not exists tifosi;

CREATE USER IF NOT EXISTS 'tifosi'@'localhost' IDENTIFIED BY 'tifosiadmin';
grant all privileges on tifosi.* to 'tifosiadmin';
flush privileges;

use tifosi;
```

Maintenant que le script permet la création de la base de données, il faut créer les tables de la base de données. La création d'une table se fait comme tel:

```
create table menu (
    id_menu int primary key not null,
    nom_menu varchar(45) not null,
    prix_menu float not null,
    boissonId int not null,
    foccaciaId int not null,
    foreign key(boissonId) references boisson(id_boisson),
    foreign key(foccaciaId) references foccacia(id_foccacia)
);
```

La table du bloc du script ci-dessus est celui construisant la table menu. On observe d'abord le "create table menu" qui permet de créer une table et de lui donner un nom. On a ensuite le nom des colonnes composant la table avec leur spécificité. Par exemple, le id_menu contient l'attribut "primary key" qui va le définir comme la key de la table. Les attributs int, varchar et float indiquent le type de données. Enfin, les relations sont créées à l'aide de "foreign key" qui vont permettre de se lier à la primary key d'une autre table comme ici avec la table boisson et la table foccacia. Dans le cas de la relation avec la table foccacia donc une relation One-to-Many, rien de plus se fera par rapport à la relation comme le montre

le bloc de code ci-dessous.

```
create table focaccia (  
    id_focaccia int primary key auto_increment not null,  
    nom_focaccia varchar(45) not null,  
    prix_focaccia float not null  
);
```

Par ailleurs, dans le cas d'une relation Many-To-Many, une troisième table est nécessaire pour créer la relation entre les deux tables comme dans le cas avec la relation avec la table "client". Cette table est une table ternaire. Elle permet de faire une relation Many-To-Many entre deux tables, et aussi d'ajouter des informations comme ici le jour où le client a commandé ce menu.

```
create table ClientMenu (  
    clientId int not null,  
    menuId int not null,  
    foreign key(clientId) references client(id_client),  
    foreign key(menuId) references menu(id_menu),  
    jour DATE  
);
```

2. Précisez les moyens utilisés :

Pour ce projet, j'ai utilisé le langage SQL ainsi que Wamp Server pour créer la base de données et créer un backup de celle-ci.

3. Avec qui avez-vous travaillé ?

J'ai travaillé seul sur ce projet.

4. Contexte

Nom de l'entreprise, organisme ou association ► Restaurant Tifosi

Chantier, atelier, service ►

DOSSIER PROFESSIONNEL ^(DP)

Période d'exercice - Du : 01/11/2024 au : 07/11/2024

5. Informations complémentaires (facultatif)

Activité-type 2 Développer le backend d'une application web

Exemple n° 2 - Développer le backend d'une application web

1. Décrivez les tâches ou opérations que vous avez effectuées, et dans quelles conditions :

Dans le cadre du développement de la boutique en ligne Sac'Azura, j'ai utilisé le framework NestJs. Celui-ci utilise un ORM qui permet de lier la base de données à la partie code de l'API. Dans le cas de NestJs, l'ORM utilisé est TypeOrm. Cet ORM va permettre dans un premier temps de créer la base de données à l'aide de migration. Pour faire cela, il faut d'abord créer un fichier, le ormconfig.ts.

```
import { DataSourceOptions, DataSource } from "typeorm";
import * as dotenv from 'dotenv'

dotenv.config()
const isCompiled = __dirname.includes('dist')

export const dataSourceOptions: DataSourceOptions = {
  type: 'mysql',
  host: process.env.DB_HOST,
  port: Number(process.env.DB_PORT) || 3306,
  username: process.env.DB_USER,
  password: process.env.DB_PASSWORD || '',
  database: process.env.DB_DATABASE,
  bigNumberStrings: true,
  multipleStatements: true,
  synchronize: false, //false in prod
  logging: true,
```

```
    entities: [ isCompiled ? 'dist/entities/*.entity{.ts,.js}' :  
'src/entities/*.entity{.ts,.js}'],  
    migrations: [isCompiled ? 'dist/database/migrations/*{.js,.ts}' :  
'src/database/migrations/*{.ts,.js}'],  
    migrationsRun: false  
}  
  
export default new DataSource(dataSourceOptions)
```

Ce fichier est la porte d'entrée de l'ORM. Je lui ai indiqué le type de base de données ainsi que l'url et les identifiants pour se connecter à celle que j'ai créé en amont. J'ai aussi indiqué le dossier qui contiendra les migrations et les entités. Les dernières options sont pour le formatage ou la connexion.

Pour créer les tables de la base de données, j'ai créé plusieurs entités comme celle ci-dessous.

```
@Entity('Address')  
export class AddressEntity {  
    @PrimaryGeneratedColumn({ type: 'int' })  
    id!: number  
  
    @Column({ type: "enum", enum: AddressType})  
    type!: AddressType  
  
    @Column({ length: 255, nullable: false})  
    street!: string  
  
    @Column({ length: 255 })  
    additional!: string  
  
    @Column({ length: 20, nullable: false })  
    zipcode!: string;  
  
    @Column({ length: 255, nullable: false})  
    city!: string;  
  
    @ManyToOne(  
        () => UserEntity,  
        (user) => user.addresses,  
        {onDelete: "CASCADE"}  
    )  
    @Index()
```

```
@Exclude()  
user!: UserEntity;  
  
@OneToMany(() => OrderEntity, (order) => order.delivery_address)  
orders!: OrderEntity[]  
}
```

Les décorateurs `@Entity` et `@Column` sont la base d'une entité. Elles vont indiquer le nom de la table, mais aussi le nom de chaque colonne avec le type de données et ses spécificités tel que la longueur de la chaîne de caractère ou encore si la colonne est nullable dans le cas présent. Les relations se créent aussi à l'aide des décorateurs `@Many-To-One`, `@One-To-Many` et `@Many-To-Many`. Un autre décorateur important est le `@PrimaryGeneratedColumn`. Celui-ci va tout simplement indiquer quelle colonne va être la primary key de la table. Dans le cas présent, il s'agit de l'id de la catégorie.

Les entités étant faites, j'ai ensuite utilisé les migrations pour créer les tables avec leur colonnes. J'ai dans un premier temps créé deux scripts: un script générant les fichiers de migrations et un les utilisant.

```
"migration:generate": "npm run typeorm -- migration:generate",  
"migration:run": "npm run typeorm -- migration:run",
```

Le premier script va créer le fichier de migration. Le script `typeorm` va uniquement build l'application et récupérer le fichier `ormconfig.js` présent dans le dossier du build de l'application. Après avoir créé le fichier de migration en lui indiquant bien le chemin vers le dossier défini plus haut dans la configuration de l'ORM, on peut utiliser la seconde commande pour effectuer la migration.

Pour accéder à la base de données depuis l'API, j'ai décidé de ne pas créer de fichier repository mais de directement utiliser le décorateur `@InjectRepository` dans mes services via le constructor comme dans l'exemple avec l'entité `Category`.

```
constructor(  
  @InjectRepository(CategoryEntity)  
  private readonly categoryRepository: Repository<CategoryEntity>  
) {}
```

Ce morceau de code fait partie du service que j'ai créé pour gérer les catégories permettant de faire les actions du CRUD tel que récupérer d'une ou de toutes les catégories, en créer une, puis la supprimer ou la modifier.

```
import { Injectable, NotFoundException } from "@nestjs/common";
import { InjectRepository } from "@nestjs/typeorm";
import { Repository } from "typeorm";
import { CategoryEntity } from "../../entities/categories.entity";
import { AddCategoryDto } from "../../shared/dtos/category/addCategory.dto";
import { UpdateCategoryDto } from "../../shared/dtos/category/updateCategory.dto";

@Injectable()
export class CategoryService {

  constructor(
    @InjectRepository(CategoryEntity)
    private readonly categoryRepository: Repository<CategoryEntity>
  ) {}

  async getAllCategory() {
    const categories = await this.categoryRepository.find({
      relations: ['subcategories']
    });

    return categories;
  }

  async findCategoryById(categoryId: number) {
    const category = await this.categoryRepository.findOne({ where: { id:
categoryId } });

    if (!category) {
      throw new NotFoundException;
    }

    return category;
  }

  async addCategory(addCategoryDto: AddCategoryDto) {
    const newCategory = await this.categoryRepository.create({
      ...addCategoryDto
    });
  }
}
```

```
        await this.categoryRepository.save(newCategory)

        return newCategory
    }

    async updateCategory(categoryId: number, updateCategoryDto: UpdateCategoryDto) {
        const category = await this.categoryRepository.findOne({ where: {
id:categoryId}})

        if(!category){
            throw new NotFoundException
        }

        Object.assign(category, updateCategoryDto)

        return this.categoryRepository.save(category)
    }

    async deleteCategory(categoryId: number) {
        const category = await this.categoryRepository.findOne({ where: {id:
categoryId}})

        if(!category){
            throw new NotFoundException
        }

        return this.categoryRepository.remove(category)
    }
}
```

Pour accéder à ce service, j'ai créé pour chaque feature un controller qui va faire office de routeur pour chacune d'elle. Chaque décorateur `@Get()`, `@Post()`, `@Patch()` ou `@Delete()` est lié à une des méthodes du service associé.

```
import { Controller, Get, Param, ParseIntPipe, Post, Body, UseGuards, Patch, Delete } from
"@nestjs/common";
import { CategoryService } from "../category.service";
import { JwtAuthGuard } from "../../auth/guards/jwt-auth.guard";
import { AdminGuard } from "../../auth/guards/admin.guard";
import { AddCategoryDto } from "../../../shared/dtos/category/addCategory.dto";
import { UpdateCategoryDto } from "../../../shared/dtos/category/updateCategory.dto";
```

```
@Controller('category')
export class CategoryController{
  constructor(
    private readonly categoryService: CategoryService
  ) {}

  @Get('')
  async getAllCategory(){
    return this.categoryService.getAllCategory()
  }

  @Get('/:categoryId')
  async findCategoryById(
    @Param('categoryId', ParseIntPipe) categoryId: number
  ) {
    return this.categoryService.findCategoryById(categoryId)
  }

  @Post('add-category')
  @UseGuards(JwtAuthGuard, AdminGuard)
  async addCategory(
    @Body() addCategoryDto: AddCategoryDto
  ){
    return this.categoryService.addCategory(addCategoryDto)
  }

  @Patch('update-category/:categoryId')
  @UseGuards(JwtAuthGuard, AdminGuard)
  async updateCategory(
    @Param('categoryId', ParseIntPipe) categoryId: number,
    @Body() updateCategoryDto: UpdateCategoryDto
  ){
    return this.categoryService.updateCategory(categoryId, updateCategoryDto)
  }

  @Delete('delete-category/:categoryId')
  @UseGuards(JwtAuthGuard, AdminGuard)
  async deleteCategory(
    @Param('categoryId', ParseIntPipe) categoryId: number
  ) {
    return this.categoryService.deleteCategory(categoryId)
  }
}
```

J'ai utilisé plusieurs autres décorateurs comme le `@Param()` qui permet de récupérer des paramètres dans l'url comme un id, ou encore le décorateur `@Body()` qui permet de récupérer les informations d'un

formulaire après qu'un DTO les ait vérifié en se basant sur des règles définies comme dans le code ci-dessous pour la création d'une catégorie.

```
import { IsNotEmpty, IsString } from "class-validator";

export class AddCategoryDto {
  @IsNotEmpty()
  @IsString()
  name!: string

  @IsNotEmpty()
  @IsString()
  sku_code!: string
}
```

Un autre décorateur que j'ai utilisé est le `@UseGuard()`. Il permet de bloquer l'accès à une route si une ou plusieurs conditions ne sont pas respectées. Par exemple, dans le cas de la création de catégorie, j'ai défini un `JwtAuthGuard` qui va s'assurer à l'aide de la librairie `passport-jwt` si l'utilisateur est connecté, et un second guard que j'ai créé pour vérifier si l'utilisateur est un administrateur.

```
import { CanActivate, ExecutionContext, ForbiddenException, Injectable } from
"@nestjs/common";

@Injectable()
export class AdminGuard implements CanActivate {

  canActivate(context: ExecutionContext): boolean {
    const request = context.switchToHttp().getRequest()
    const user = request.user

    if(!user || !user.isAdmin) {
      throw new ForbiddenException("Route interdite")
    }

    return true
  }
}
```

J'ai appliqué cette méthodologie pour chaque feature de ce projet.

DOSSIER PROFESSIONNEL ^(DP)

2. Précisez les moyens utilisés :

Pour ce travail, j'ai utilisé NestJs avec son ORM, TypeORM. J'ai aussi utilisé les librairies Passport-jwt pour gérer l'authentification et la persévérance dans le temps de celle-ci. Pour les DTO, les librairies class-transformer et class-validator ont été utilisées.

3. Avec qui avez-vous travaillé ?

J'ai travaillé seul sur cette partie du projet.

4. Contexte

Nom de l'entreprise, organisme ou association ▶ Sac'Azura

Chantier, atelier, service ▶

Période d'exercice ▶ Du : 22/10/2025 au : actuellement en cours

5. Informations complémentaires *(facultatif)*

Activité-type 2 Développer le backend d'une application web

Exemple n° 3 - Intégrer une solution de paiement dans une application web

1. Décrivez les tâches ou opérations que vous avez effectuées, et dans quelles conditions :

Le projet Sac'Azura étant une boutique en ligne, il m'a fallu intégrer une solution de paiement. Mon choix s'est alors porté sur Stripe.

Avant de créer la session de paiement, j'ai créé dans le service payment.service.ts une méthode pour préparer le paiement. Celle-ci va d'abord récupérer l'utilisateur faisant l'achat ainsi que ses adresses de livraison et de paiement. Elle va ensuite récupérer les produits dans le panier pour les mettre dans un tableau. Enfin, elle va vérifier si un code de promotion existe, est actif et si la condition d'utilisation est respectée.

```
const currentUser = await this.userRepository.findOneBy({ id: user.id });

if (!currentUser) {
    throw new NotFoundException("Utilisateur introuvable");
}

if (!currentUser.isVerified) {
    throw new ForbiddenException("Votre compte doit être vérifié pour effectuer un achat.");
}

const deliveryAddress = await this.addressRepository.findOneBy({
    id: cartDto.delivery_address_id,
    user: { id: user.id }
});

const billingAddress = await this.addressRepository.findOneBy({
    id: cartDto.billing_address_id,
    user: { id: user.id }
});

if (!deliveryAddress || !billingAddress) {
    throw new InternalServerErrorException("Les adresses n'appartiennent pas à cet utilisateur");
}
```

```
let totalAmount = 0;

const order = new OrderEntity();
order.user = user;
order.status = OrderStatus.PENDING;
order.items = [];
order.delivery_address = deliveryAddress;
order.billing_address = billingAddress;

for (const item of cartDto.items) {
    const variant = await this.stockRepository.findOne({ where: { sku:
item.sku }, relations: ['product'] });

    if (!variant) {
        throw new NotFoundException("Le produit n'a pas été trouvé");
    }

    if (!variant.product) {
        throw new NotFoundException("Le produit n'a pas été trouvé");
    }

    if (variant.quantity < item.quantity) {
        throw new BadRequestException('Le stock est inférieur à la quantité
commandée');
    }

    const orderItem = new OrderItemEntity();
    orderItem.stock = variant;
    orderItem.quantity = item.quantity;
    orderItem.priceAtPurchase = variant.product.price;

    order.items.push(orderItem);

    totalAmount += variant.product.price * item.quantity;
}

// Promotion code management ready to be implemented in the frontend
if (cartDto.promotion_code && cartDto.promotion_code !== "AUCUN" &&
cartDto.promotion_code.trim() !== "") {
    const promotionCode = await this.promotionRepository.findOne({ where: {
name: cartDto.promotion_code } });

    if (!promotionCode) {
```

```
        throw new BadRequestException("Le code de promotion renseigné n'existe pas ou a expiré");
    }

    if (promotionCode.minAmount >= totalAmount) {
        throw new BadRequestException("Vous n'avez pas atteint la valeur minimum pour utiliser ce code promotionnel")
    } else {
        if (promotionCode.promotionType === "percentage") {
            const promotionValue = promotionCode.percentageValue;
            totalAmount = totalAmount - (totalAmount * promotionValue) / 100;
        } else if (promotionCode.promotionType === 'fixed_amount') {
            const promotionValue = promotionCode.fixedValue;
            totalAmount = totalAmount - promotionValue;
        }
    }
}

order.totalAmount = totalAmount;

const savedOrder = await this.orderRepository.save(order);

return this.createCheckoutSession(savedOrder.id, currentUser.id, order.items, currentUser.mail);
}
```

Après avoir vérifié les données, j'ai créé la session de paiement. J'ai dans un premier temps utilisé un `.map` pour ajouter à la constante `line_items` tous les produits commandés en vérifiant qu'ils existent bien. On rentre ensuite dans un `try/catch` qui va ouvrir la session de paiement Stripe, puis retourner un succès ou un échec en suivant les urls désignées.

```
async createCheckoutSession(orderId: number, userId: number, items: OrderItemEntity[], userEmail: string) {
    const line_items = items.map(item => {
        if (!item.stock.product?.name) {
            throw new InternalServerErrorException('Données du produit manquantes')
        }
    })

    return {
        price_data: {
```

```
        currency: 'eur',
        product_data: {
            name: item.stock.product.name,
            description: `Modèle: ${item.stock.sku}`
        },
        unit_amount: Math.round(item.priceAtPurchase * 100)
    },
    quantity: item.quantity
}
});

try {
    const session = await this.stripe.checkout.sessions.create({
        payment_method_types: ["card"],
        line_items,
        mode: 'payment',
        customer_email: userEmail,
        success_url: 'http://localhost:3000/payment/payment_success',
        cancel_url:
`http://localhost:3000/payment/payment_failed/${orderId}/${userId}`,

        metadata: {
            orderId: orderId.toString(),
            userId: userId.toString()
        }
    });

    return { url: session.url };
} catch (error) {
    console.error("Error creating session: ", error);
    throw new InternalServerErrorException('failed to create checkout
session');
}
}
```

En cas d'échec durant le paiement comme la session qui se fermerait ou l'utilisateur qui mettrait trop de temps à payer avant que la session n'expire, un mail lui est envoyé.

```
async paymentFailed(orderId: number, userId: number) {
    const order = await this.orderRepository.findOne({ where: { id: orderId, user:
{ id: userId } }, relations: ['user'] });
```

```
if (!order) {
    throw new ForbiddenException('Cette commande ne vous appartient pas');
}

if (order && order.status === OrderStatus.PENDING) {
    order.status = OrderStatus.CANCELLED;
    await this.orderRepository.save(order);
}

console.log('la commande mise a jour est:', order);

const mail = order.user.mail;
await this.sendMailPaymentFail(orderId, mail);
}
```

Un mail d'échec peut aussi être envoyé dans le cas d'un échec de paiement.

En cas de succès, j'ai aussi créé un webhook qui va récupérer l'information du succès de la session de paiement (et non juste du succès de paiement). Ce webhook va alors modifier en base de données les informations sur le statut de la commande ainsi les stocks des produits achetés, tout ça en utilisant les métadonnées défini dans la session stripe avec l'id de l'utilisateur et l'id de la commande.

```
async constructEventWebhook(req: any, res: any, signature: string) {
    const endpointSecret = process.env.SECRET_WEBHOOK_KEY;
    if (!endpointSecret) {
        throw new NotFoundException("Le webhook ne fonctionne pas");
    }

    let event: Stripe.Event;
    try {
        event = this.stripe.webhooks.constructEvent(
            req.rawBody,
            signature,
            endpointSecret
        );
    } catch (error) {
        if (error instanceof Error) {
            return res.status(401).send(`webhook error: ${error.message}`);
        } else {
            return res.status(500).send('An unknown error occurred');
        }
    }
}
```

```
}

if (event.type === 'checkout.session.completed') {
    const session = event.data.object as Stripe.Checkout.Session;
    const metadata = session.metadata;

    if (!metadata) {
        throw new InternalServerErrorException('Les métadonnées n\'existent pas');
    }

    const orderId = Number(metadata.orderId);
    const userId = Number(metadata.userId);

    try {
        await this.dataSource.transaction(async (transactionalEntityManager) => {
            const order = await transactionalEntityManager.findOne(OrderEntity, {
                where: { id: orderId },
                relations: ['items', 'items.stock']
            });

            if (!order || order.status !== OrderStatus.PENDING) {
                return;
            }

            order.status = OrderStatus.PAID;
            await transactionalEntityManager.save(order);

            for (const line of order.items) {
                const currentStock = line.stock;

                if (currentStock.quantity < line.quantity) {
                    throw new InternalServerErrorException(`Stock insuffisant pour l'item ${currentStock.sku}`)
                }

                currentStock.quantity -= line.quantity;
                await transactionalEntityManager.save(currentStock);

                console.log(`Transaction réussie. Commande ${orderId} payée et stock déduits.`);
            }
        });
    }
}
```

```
        }
        const user = await this.userRepository.findOne({ where: { id:
userId } }));

        if (!user) {
            throw new InternalServerErrorException(`L'utilisateur avec
l'id ${userId} n'existe pas`);
        }
        const mail = user.mail;
        await this.sendMailPaymentSuccess(mail, orderId);
    });
} catch (error) {
    console.error('Échec de la transaction');
}

} else if (event.type === 'checkout.session.expired') {
    const session = event.data.object as Stripe.Checkout.Session;
    const metadata = session.metadata;

    if (!metadata) {
        throw new InternalServerErrorException('Les métadonnées n\'existent
pas');
    }

    const orderId = Number(metadata.orderId);
    const order = await this.orderRepository.findOne({ where: { id: orderId }
});

    if (order && order.status === OrderStatus.PENDING) {
        order.status = OrderStatus.CANCELLED;
        await this.orderRepository.save(order);
    }
}

return res.status(200).json({ received: true });
}
```

2. Précisez les moyens utilisés :

J'ai utilisé le framework NestJs pour ce projet, et j'ai utilisé l'API Stripe pour gérer la partie paiement. La librairie nodemailer, quant à elle, a été utilisée pour l'envoi de mail.

DOSSIER PROFESSIONNEL ^(DP)

3. Avec qui avez-vous travaillé ?

J'ai travaillé seul sur cette partie du projet.

4. Contexte

Nom de l'entreprise, organisme ou association ▶ Sac'Azura

Chantier, atelier, service ▶

Période d'exercice ▶ Du : 22/10/2025 au : actuellement en cours

5. Informations complémentaires *(facultatif)*

Titres, diplômes, CQP, attestations de formation

(facultatif)

Intitulé	Autorité ou organisme	Date
Earth and Planetary Master Degree	Laboratoire de Planétologie et Géodynamique, Faculté des sciences, Nantes	2023
Licence Science de la Terre et des Planètes, environnements	Faculté des sciences, Le Mans Université	2021

DOSSIER PROFESSIONNEL ^(DP)

Déclaration sur l'honneur

Je soussigné(e) [prénom et nom] **Mr.Barbey Mathieu** ,
déclare sur l'honneur que les renseignements fournis dans ce dossier sont exacts et que je suis
l'auteur(e) des réalisations jointes.

Fait à **La Loupe**

le Cliquez ici pour choisir une date

pour faire valoir ce que de droit.

Signature :

Documents illustrant la pratique professionnelle

(facultatif)

Intitulé
Cliquez ici pour taper du texte.

DOSSIER PROFESSIONNEL ^(DP)

ANNEXES

(Si le RC le prévoit)